

マージソート

「クイックソート」と並んで、非常に効率的で重要なアルゴリズムが「マージソート（併合ソート）」です。

クイックソートが「適当な基準で分ける」のに対し、マージソートは「きっちり半分に分け、あとで綺麗に合体させる」という、非常に几帳面なアルゴリズムです。

1. マージソートの基本戦略

- マージソートは、大きく分けて2つのステップを繰り返します。
- 1. **分割 (Divide):** データを要素が1つになるまで、ひたすら半分に分割し続けます。
 - 2. **併合 (Merge):** 分割されたデータを、順番を守りながら合体（マージ）させていきます。

2. 具体的な手順（4, 2, 5, 1 を並べ替える）

比較	数値	図
	4	####
	2	##
	5	#####
	1	#

ステップ1：バラバラに解体する

- [4, 2, 5, 1] を半分に → [4, 2] と [5, 1]

比較	数値	図
	4	####
	2	##
比較	数値	図
	5	#####
	1	#

- さらに半分に → [4] [2] [5] [1]

比較	数値	図
	4	####

比較	数値	図
----	----	---

2 ##

比較	数値	図
----	----	---

5 #####

比較	数値	図
----	----	---

1 #

(これで全員が1人ぼっちになりました。1つの要素は、それ自体が「整列済み」と言えます)

ステップ2：順番を守って合体させる

ここからが「マージ」の本領発揮です。

① 比較 ② 合体

の順で作業します。

- **[4]** と **[2]** を合体：2つを比べて小さい順に並べる → **[2, 4]**

① 比較

比較	数値	図
----	----	---

* 4 #####

比較	数値	図
----	----	---

* 2 ##

② 合体

比較	数値	図
----	----	---

2<4 なのでこちらが上 2 ##

4 #####

- **[5]** と **[1]** を合体：2つを比べて小さい順に並べる → **[1, 5]**

① 比較

比較	数値	図
----	----	---

* 5 #####

比較	数値	図
----	----	---

* 1 #

② 合体

比較	数値	図
1<5 なのでこちらが上	1	#
	5	#####

- **[2, 4]** と **[1, 5]** を合体：

1. それぞれの先頭（2と1）を比べる → **1** を採用

① 比較

比較	数値	図
*	2	##
	4	####
比較	数値	図
*	1	#
	5	#####

② 合体

比較	数値	図
	1	#

残りは

比較	数値	図
	2	##
	4	####
比較	数値	図
	5	#####

2. 残った先頭（2と5）を比べる → **2** を採用

① 比較

比較	数値	図
*	2	##
	4	####
比較	数値	図
*	5	#####

② 合体

比較	数値	図
	1	#
	2	##

残りは

比較	数値	図
	4	####
比較	数値	図
	5	#####

3. 残った先頭（4と5）を比べる → **4** を採用

① 比較

比較	数値	図
*	4	####
比較	数値	図
*	5	#####

② 合体

比較	数値	図
	1	#
	2	##
	4	####

残りは

比較	数値	図
	5	#####

4. 最後に残った **5** を採用

② 合体

比較	数値	図
	1	#
	2	##
	4	####

比較 数値 図

5 #####

- 完成：[1, 2, 4, 5]

3. マージソートのメリット・デメリット

項目	特徴
安定性	非常に安定している。クイックソートと違い、データの並び順が悪くても速度が落ちません。
計算量	常に。どんなにバラバラなデータでも、一定の速さを保証します。
弱点	メモリを多く使う。合体作業をするために、一時的に別の「作業スペース（予備の配列）」が必要になります。

4. クイックソートとの違い

- クイックソート：* 最初に「仕分け（パーティション）」をして、あとで繋げる。
- 準備は大変だけど、合体は楽。
- 「現場で仕分けるタイプ」
- マージソート：* とにかくバラバラにしてから、合体させるときに整列する。
- 準備は楽だけど、合体（マージ）が一番重要。
- 「後からきっちりまとめるタイプ」

実装

マージソートの実装は、クイックソートと同様に **再帰** を使用しますが、最大の特徴は「バラバラにしたものを合体（マージ）させる際に整列させる」という点です。

合体作業（マージ）を行う際、一時的にデータを避難させるための**「作業用配列」**が必要になることが、他のソートとの大きな違いです。

疑似言語によるマージソートの実装

```
/* グローバル変数として作業用の配列を定義（最大数100を想定） */
整数型[]: temp ← 100要素の配列

/* 2つの整列済み範囲を1つに合体させる関数 */
○ merge(参照型: 整数型[]: arr, 整数型: left, 整数型: mid, 整数型: right)
```

```

整数型: i ← left      /* 左側グループの先頭 */
整数型: j ← mid + 1    /* 右側グループの先頭 */
整数型: k ← 1          /* 作業用配列の添字（疑似言語仕様により添え字は1からスタート）
*/

/* 左右の要素を比較して、小さい方を順に作業用配列へコピー */
while (i ≤ mid and j ≤ right)
    if (arr[i] ≤ arr[j])
        temp[k] ← arr[i]
        i ← i + 1
    else
        temp[k] ← arr[j]
        j ← j + 1
    endif
    k ← k + 1
endwhile

/* 左側に残った要素をコピー */
while (i ≤ mid)
    temp[k] ← arr[i]
    i ← i + 1
    k ← k + 1
endwhile

/* 右側に残った要素をコピー */
while (j ≤ right)
    temp[k] ← arr[j]
    j ← j + 1
    k ← k + 1
endwhile

/* 作業用配列（1から k - 1 まで格納された）から元の配列へ書き戻す */
/* ループ用の変数: m */
for (整数型: m を 1 から k - 1 まで 1 ずつ増やす)
    /* C言語の left + i (0スタート) は、1スタートの世界では left + m - 1 になる */
    arr[left + m - 1] ← temp[m]
endfor

/* マージソートの本体（分割を担当） */
○ mergeSort(参照型: 整数型[]: arr, 整数型: left, 整数型: right)
    if (left < right)
        /* 真ん中の位置を計算（整数除算、端数切り捨て） */
        整数型: mid ← (left + right) ÷ 2

        mergeSort(arr, left, mid)      /* 左半分を分割 */
        mergeSort(arr, mid + 1, right) /* 右半分を分割 */

        merge(arr, left, mid, right)    /* 最後に合体（マージ） */
    endif

/* メイン処理（テスト実行用） */
○ main()
    /* 試験仕様に合わせて1番目から4番目の要素として初期化 */

```

```

整数型[]: data ← [4, 2, 5, 1]
整数型: n ← 4

/* マージソートの呼び出し（添字は 1 から n まで） */
mergeSort(data, 1, n)

```

C言語によるマージソートの実装

```

#include <stdio.h>

// 作業用の配列（データの最大数に合わせて用意）
int temp[100];

// 2つの整列済み範囲を1つに合体させる関数
void merge(int arr[], int left, int mid, int right) {
    int i = left;        // 左側グループの先頭
    int j = mid + 1;     // 右側グループの先頭
    int k = 0;           // 作業用配列の添字

    // 左右の要素を比較して、小さい方を順に作業用配列へコピー
    while (i <= mid && j <= right) {
        if (arr[i] <= arr[j]) {
            temp[k++] = arr[i++];
        } else {
            temp[k++] = arr[j++];
        }
    }

    // 左側に残った要素をコピー
    while (i <= mid) temp[k++] = arr[i++];
    // 右側に残った要素をコピー
    while (j <= right) temp[k++] = arr[j++];

    // 作業用配列から元の配列へ書き戻す
    for (i = 0; i < k; i++) {
        arr[left + i] = temp[i];
    }
}

// マージソートの本体（分割を担当）
void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = (left + right) / 2; // 真ん中を計算

        mergeSort(arr, left, mid);    // 左半分を分割
        mergeSort(arr, mid + 1, right); // 右半分を分割

        merge(arr, left, mid, right); // 最後に合体（マージ）
    }
}

```

```
int main() {
    int data[] = {4, 2, 5, 1};
    int n = 4;

    printf("ソート前: ");
    for (int i = 0; i < n; i++) printf("%d ", data[i]);
    printf("\n");

    mergeSort(data, 0, n - 1);

    printf("ソート後: ");
    for (int i = 0; i < n; i++) printf("%d ", data[i]);
    printf("\n");

    return 0;
}
```

ポイント解説

1. 「分割」と「統合」の役割分担

- `mergeSort` 関数：配列を真ん中でバツサリ切り続ける役割です。
- `merge` 関数：バラバラになった数字を、**「2つの整列済みリストを1つにまとめる」**というルールで綺麗に並べ直す役割です。

2. 一時的な作業スペース (`temp`)

- マージソートは、合体作業中に元の配列の値を上書きしてしまうため、一時的に避難させるための「作業台」が必要です。これがメモリを余分に使う理由です。

3. 比較のロジック

- `while` ループの中で、左の山と右の山の「先頭」だけを見比べて、小さい方を採用しています。これはトランプの山を2つ作り、上のカードをめくりながら小さい方を新しい山に積んでいく動きと同じです。

安定性の証明

「マージソートは、なぜ同じ数字の順番が入れ替わらない（安定な）の？」

その理由：

- `if (arr[i] <= arr[j])`
- ここで `<=` (以下) としているため、左側にあった同じ値が優先的に採用されます。これが「安定性」を支える重要な一行であることを伝え、アルゴリズムの繊細さが伝わります。